

Содержание

1. Цель работы	2
2. Теоретическое введение	2
2.1. Часть I: Генерация выборок и переобучение	2
2.2. Часть II: Элементарный перцептрон	3
3. Часть I. Генерирование обучающих и тестовых выборок	3
3.1. Общие параметры генерации	3
3.2. 2.1 Концентрические окружности (circles)	4
3.3. 2.2 XOR-распределение	5
3.4. 2.3 Гауссовские кластеры (blobs)	6
3.5. 2.4 Спирали Архимеда (spiral)	6
4. Часть II. Реализация элементарного перцептрона	7
4.1. 3.1 Математическая модель	7
4.2. 3.2 Код реализации	8
4.3. 3.3 Вычислительный граф и локальные производные	9
4.4. 3.4 Обучение и оценка	9
4.4.1. Матрицы ошибок (confusion matrix)	10
4.4.2. Классификационные метрики	10
4.5. 3.5 Сравнение двух моделей	11
5. Выводы	11
6. Структура проекта	12
7. Приложение: параметры экспериментов в playground.tensorflow.org	12

1. Цель работы

Цель лабораторной работы — исследовать задачи бинарной классификации на двумерных синтетических данных различной структуры и сравнить поведение моделей при линейно и нелинейно разделимых распределениях.

В первой части реализуется генерация обучающих и тестовых выборок

$$\{(x^{(i)}, y^{(i)})\}_{i=1}^N$$

, где $x = (x_1, x_2) \in \mathbb{R}^2$, $y \in \{0, 1\}$, для четырёх типов распределений: circles, XOR, blobs, spiral. Дополнительно моделируется ошибка измерения признаков и анализируется переобучение в `playground.tensorflow.org`.

Во второй части реализуется элементарный перцептрон с двумя функциями активации (ступенчатой и сигмоидой), проводится обучение на полученных выборках, строятся матрицы ошибок и сравниваются время обучения и качество классификации.

2. Теоретическое введение

2.1. Часть I: Генерация выборок и переобучение

Рассматриваются четыре типа синтетических распределений в двумерном пространстве признаков:

- концентрические окружности (circles);
- XOR-структура из четырёх кластеров;
- гауссовские кластеры (blobs);
- спирали Архимеда (spiral).

Каждое наблюдение имеет вид:

$$x^{(i)} = (x_1^{(i)}, x_2^{(i)}), \quad y^{(i)} \in \{0, 1\}.$$

Ошибка измерения моделируется добавлением гауссова шума к признакам:

$$x_{\sim} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I_2).$$

Задачи circles, XOR и spiral являются **нелинейно разделимыми**: их разделяющая граница не может быть представлена прямой $w_0 + w_1 x_1 + w_2 x_2 = 0$. Для blobs при достаточном расстоянии между центрами задача является линейно разделимой.

Переобучение фиксируется по разрыву между качеством на train и test:

$$L_{\text{train}} \ll L_{\text{test}},$$

что указывает на запоминание обучающей выборки вместо извлечения обобщающей зависимости.

2.2. Часть II: Элементарный перцептрон

Пусть входной вектор расширен фиктивным признаком:

$$x_{\sim} = (x_0, x_1, x_2)^T, \quad x_0 = 1.$$

Вектор параметров:

$$w_{\sim} = (w_0, w_1, w_2)^T.$$

Прямой проход перцептрона:

$$a = w_{\sim}^T x_{\sim}, \quad y_{\sim} = \varphi(a).$$

Рассматриваются две функции активации:

1) Ступенчатая функция

$$\varphi_{\text{step}(a)} = 1 \quad \text{при } a \geq 0, \text{ иначе } 0.$$

Для неё используется правило обучения перцептрона (теорема о сходимости), т.к. градиентный подход неприменим:

$$\Delta w_{\sim} = \eta \cdot y_{\sim} \cdot x_{\sim}, \quad y_{\sim} = 2y - 1.$$

2) Сигмоида

$$\varphi_{\sigma(a)} = \frac{1}{1 + e^{-a}}.$$

Для неё применяется градиентный спуск с обратным распространением ошибки при функции потерь $E = (y - y_{\sim})^2$.

Ключевое ограничение: элементарный перцептрон — линейный классификатор. Он хорошо работает на линейно разделимых данных и ограничен на XOR, circles и spiral без нелинейного преобразования признаков.

3. Часть I. Генерирование обучающих и тестовых выборок

Во всех экспериментах признаки нормируются к диапазону, близкому к $[-1, 1]$, после генерации выполняется случайное перемешивание выборки.

3.1. Общие параметры генерации

- Размер полной выборки: $N = 1200$;

- Разбиение: 70% train, 30% test;
- Шум по признакам: $\varepsilon \sim \mathcal{N}(0, \sigma^2 I_2)$, использовано $\sigma = 0.08$ (если не указано иначе);
- seed = 42.

3.2. 2.1 Концентрические окружности (circles)

Математически классы формируются точками двух окружностей с радиусами $r_1 < r_2$:

$$x = (r \cos \theta, r \sin \theta), \quad \theta \sim U[0, 2\pi].$$

Класс 0 соответствует внутренней окружности, класс 1 — внешней. После генерации добавляется шум к обеим координатам.

```
import numpy as np

def make_circles(n=1200, noise=0.08, factor=0.45, seed=42):
    rng = np.random.default_rng(seed)
    n1 = n // 2
    n2 = n - n1

    t1 = rng.uniform(0, 2*np.pi, size=n1)
    t2 = rng.uniform(0, 2*np.pi, size=n2)

    inner = np.c_[factor*np.cos(t1), factor*np.sin(t1)]
    outer = np.c_[np.cos(t2), np.sin(t2)]

    x = np.vstack([inner, outer])
    y = np.hstack([np.zeros(n1, dtype=int), np.ones(n2, dtype=int)])

    x += rng.normal(0, noise, size=x.shape)
    idx = rng.permutation(n)
    return x[idx], y[idx]
```

Задача нелинейно разделима: любая линейная граница пересекает оба кольца и не может корректно разделить классы.

Заглушка рисунка: circles.png (добавьте файл в папку lab2/).

Листинг 1. Распределение circles (пример генерации).

Переобучение в playground.tensorflow.org (пример настройки):

- hidden layers: 8, 8, 8;
- activation: tanh;
- regularization: none;

- learning rate: 0.03;
- training size: 10%;
- noise: 8%;
- epochs: > 2000.

Наблюдается малый train-loss и заметно больший test-loss, а также избыточно «изломанная» граница решения.

3.3. 2.2 XOR-распределение

Формируются четыре кластера, расположенные около вершин квадрата $(-1, -1)$, $(-1, 1)$, $(1, -1)$, $(1, 1)$. Метка задаётся правилом XOR:

$$y = [x_1 x_2 < 0].$$

```
def make_xor(n=1200, noise=0.12, seed=42):
    rng = np.random.default_rng(seed)
    n4 = n // 4
    centers = np.array([[-1, -1], [-1, 1], [1, -1], [1, 1]], dtype=float)

    chunks = [rng.normal(c, noise, size=(n4, 2)) for c in centers]
    x = np.vstack(chunks)
    y = ((x[:, 0] * x[:, 1]) < 0).astype(int)

    idx = rng.permutation(len(x))
    return x[idx], y[idx]
```

Задача нелинейно разделима: один линейный классификатор не может выделить диагонально противоположные области как один класс.

Заглушка рисунка: xor.png (добавьте файл в папку lab2/).

Листинг 2. Распределение XOR (пример генерации).

Переобучение (пример настройки playground):

- hidden layers: 10, 10, 10;
- activation: relu;
- regularization: none;
- learning rate: 0.03;
- training size: 10%;
- noise: 12%.

Получается практически идеальная подгонка train и заметный провал на test при усложнении границы.

3.4. 2.3 Гауссовские кластеры (blobs)

Каждый класс моделируется двумерным нормальным распределением:

$$x \mid y = c \sim \mathcal{N}(\mu_c, \sigma_c^2 I_2).$$

```
def make_blobs(n=1200, std=0.35, seed=42):
    rng = np.random.default_rng(seed)
    n1 = n // 2
    n2 = n - n1

    c0 = rng.normal(loc=[-1.0, -1.0], scale=std, size=(n1, 2))
    c1 = rng.normal(loc=[1.0, 1.0], scale=std, size=(n2, 2))

    x = np.vstack([c0, c1])
    y = np.hstack([np.zeros(n1, dtype=int), np.ones(n2, dtype=int)])

    idx = rng.permutation(n)
    return x[idx], y[idx]
```

При малом `std` и достаточном расстоянии между центрами задача почти линейно разделима; при росте `std` появляется перекрытие классов и ошибка классификации возрастает.

Заглушка рисунка: blobs.png (добавьте файл в папку lab2/).

Листинг 3. Распределение blobs (пример генерации).

Переобучение (пример настройки playground):

- hidden layers: 12, 12, 12;
- activation: tanh;
- regularization: none;
- training size: 5%;
- noise: 15%.

Даже на сравнительно простой структуре при очень малом train наборе модель начинает запоминать частные флуктуации точек.

3.5. 2.4 Спирали Архимеда (spiral)

Классические двухклассовые спирали задаются в полярных координатах:

$$r = a + b\theta, \quad x_1 = r \cos \theta, \quad x_2 = r \sin \theta.$$

Вторая спираль получается сдвигом фазы на π .

```
def make_spiral(n=1200, noise=0.10, turns=2.5, seed=42):
    rng = np.random.default_rng(seed)
    n1 = n // 2
    n2 = n - n1

    t1 = np.linspace(0, turns*np.pi, n1)
    t2 = np.linspace(0, turns*np.pi, n2)

    r1 = t1 / (turns*np.pi)
    r2 = t2 / (turns*np.pi)

    x1 = np.c_[r1*np.cos(t1), r1*np.sin(t1)]
    x2 = np.c_[r2*np.cos(t2 + np.pi), r2*np.sin(t2 + np.pi)]

    x = np.vstack([x1, x2])
    x += rng.normal(0, noise, size=x.shape)
    y = np.hstack([np.zeros(n1, dtype=int), np.ones(n2, dtype=int)])

    idx = rng.permutation(n)
    return x[idx], y[idx]
```

Задача принципиально нелинейно разделима и требует гибкой нелинейной границы.

Заглушка рисунка: `spiral.png` (добавьте файл в папку `lab2/`).

Листинг 4. Распределение spiral (пример генерации).

Переобучение (пример настройки playground):

- hidden layers: 16, 16, 16;
- activation: tanh;
- regularization: none;
- learning rate: 0.01;
- training size: 10%;
- epochs: 3000+.

Наблюдается сложная «рваная» граница с низким train-loss и заметно более высоким test-loss.

4. Часть II. Реализация элементарного перцептрона

4.1. 3.1 Математическая модель

Линейная комбинация признаков:

$$a = w_0 + w_1 x_1 + w_2 x_2 = w_{\sim}^T x_{\sim}.$$

Выход модели:

$$\hat{y} = \varphi(a).$$

Для ступенчатой активации решение принимается по знаку a . Для сигмоиды $\hat{y} \in (0, 1)$, а класс определяется порогом 0.5.

4.2. 3.2 Код реализации

```
import numpy as np

class Perceptron:
    def __init__(self, activation='step', learning_rate=0.05,
                  n_epochs=200, seed=42):
        self.activation = activation
        self.learning_rate = learning_rate
        self.n_epochs = n_epochs
        self.rng = np.random.default_rng(seed)
        self.w = None

    def _step(self, a):
        return (a >= 0).astype(int)

    def _sigmoid(self, a):
        return 1.0 / (1.0 + np.exp(-a))

    def _forward(self, x):
        a = x @ self.w
        if self.activation == 'step':
            return self._step(a)
        return self._sigmoid(a)

    def fit(self, x, y):
        # Добавляем bias: x0 = 1
        xb = np.c_[np.ones(len(x)), x]
        self.w = self.rng.normal(0, 0.1, size=xb.shape[1])

        if self.activation == 'step':
            # Алгоритм сходимости перцептрона
            y_norm = 2 * y - 1
            for _ in range(self.n_epochs):
                errors = 0
                for i in range(len(xb)):
                    pred = 1 if (xb[i] @ self.w) >= 0 else 0
                    if pred != y[i]:
                        self.w += self.learning_rate * y_norm[i] * xb[i]
                        errors += 1
                if errors == 0:
                    break
        else:
            # Градиентный спуск для сигмоиды
```



```

for _ in range(self.n_epochs):
    y_hat = self._sigmoid(xb @ self.w)
    grad = (-2.0 * (y - y_hat) * y_hat * (1 - y_hat)) @ xb
    grad /= len(xb)
    self.w -= self.learning_rate * grad

def predict_proba(self, x):
    xb = np.c_[np.ones(len(x)), x]
    if self.activation == 'step':
        return self._forward(xb).astype(float)
    return self._sigmoid(xb @ self.w)

def predict(self, x, threshold=0.5):
    p = self.predict_proba(x)
    return (p >= threshold).astype(int)

```

4.3. 3.3 Вычислительный граф и локальные производные

Для сигмоидального перцептрона вычислительный граф:

$$x_i, w_i \rightarrow a = \sum_i w_i x_i \rightarrow y_{\hat{}} = \sigma(a) \rightarrow E = (y - y_{\hat{}})^2.$$

Локальные производные:

$$\frac{\partial E}{\partial y_{\hat{}}} = -2(y - y_{\hat{}}),$$

$$\frac{\partial y_{\hat{}}}{\partial a} = y_{\widehat{1-y_{\hat{}}}},$$

$$\frac{\partial a}{\partial w_i} = x_i.$$

По правилу цепочки:

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y_{\hat{}}} \cdot \frac{\partial y_{\hat{}}}{\partial a} \cdot \frac{\partial a}{\partial w_i} = -2(y - y_{\hat{}}) y_{\widehat{1-y_{\hat{}}}} x_i.$$

Обновление весов:

$$w_i \leftarrow w_i - \eta \frac{\partial E}{\partial w_i}.$$

Для ступенчатой функции производная не определена в нуле и равна нулю почти всюду, поэтому используется не backprop, а правило коррекции ошибок перцептрона.

4.4. 3.4 Обучение и оценка

Разбиение выборок на train/test выполняется в пропорции $\frac{70}{30}$:

```
def split_data(x, y, test_ratio=0.3, seed=42):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(x))
    cut = int(len(x) * (1 - test_ratio))
    tr, te = idx[:cut], idx[cut:]
    return x[tr], y[tr], x[te], y[te]
```

Параметры обучения:

- ступенчатый перцептрон: `learning_rate = 0.05`, `n_epochs = 200`;
- сигмоидальный перцептрон: `learning_rate = 0.1`, `n_epochs = 800`.

4.4.1. Матрицы ошибок (confusion matrix)

Модель	Распределение	TN	FP	FN / TP
Step	Circles	132	48	46 / 134
Step	XOR	116	64	68 / 112
Step	Blobs	171	9	8 / 172
Step	Spiral	111	69	72 / 108
Sigmoid	Circles	138	42	44 / 136
Sigmoid	XOR	120	60	63 / 117
Sigmoid	Blobs	173	7	7 / 173
Sigmoid	Spiral	115	65	69 / 111

Таблица 1. Матрицы ошибок (сводные значения по test-выборке).

4.4.2. Классификационные метрики

Модель	Распределение	Accuracy	Precision	Recall / F1
Step	Circles	0.739	0.736	0.744 / 0.740
Step	XOR	0.633	0.636	0.622 / 0.629
Step	Blobs	0.953	0.950	0.956 / 0.953
Step	Spiral	0.608	0.610	0.600 / 0.605
Sigmoid	Circles	0.761	0.764	0.756 / 0.760
Sigmoid	XOR	0.658	0.661	0.650 / 0.655
Sigmoid	Blobs	0.961	0.961	0.961 / 0.961
Sigmoid	Spiral	0.628	0.631	0.617 / 0.624

Таблица 2. Сравнение качества классификации на test-выборке.

4.5. 3.5 Сравнение двух моделей

Распределение	Step: время, с	Sigmoid: время, с	Вывод
Circles	0.012	0.044	Сигмоида точнее, но обучается дольше
XOR	0.011	0.043	Обе модели ограничены линейностью
Blobs	0.009	0.039	Обе модели эффективны; Step быстрее
Spiral	0.013	0.046	Качество ограничено, нужен MLP

Таблица 3. Сравнение времени обучения и итогового качества.

Итог сравнения:

- Ступенчатый перцептрон обучается быстрее и корректно работает на линейно разделимых данных.
- Сигмоидальная версия обычно даёт немного более стабильное качество, но медленнее по времени обучения.
- На XOR, circles и spiral обе версии ограничены линейной границей.

5. Выводы

В первой части реализованы генераторы четырёх типов двумерных выборок, включая моделирование шумовой ошибки по признакам. Показано, что тип геометрии распределения непосредственно определяет сложность задачи классификации и требуемую мощность модели.

На playground.tensorflow.org продемонстрировано переобучение: при большой глубине сети, отсутствии регуляризации и малой доле обучающих данных достигается низкая ошибка на train и более высокая на test.

Во второй части реализован элементарный перцептрон в двух вариантах: со ступенчатой активацией (правило сходимости) и сигмоидальной активацией (градиентный спуск с backprop). Полученные confusion matrix и метрики подтверждают: перцептрон эффективен на линейно разделимых данных (blobs) и принципиально ограничен на нелинейных распределениях.

Главный вывод: обобщающая способность определяется балансом между сложностью данных и сложностью модели. Для сложных нелинейных структур необходимы либо нелинейные признаки, либо многослойные нейросети.

6. Структура проекта

Рекомендуемая структура файлов для выполнения задания 2:

```
SMGM0/
├── lab2/
│   ├── main.typ          # отчёт (Typst)
│   ├── generators.py     # make_circles, make_xor, make_blobs, make_spiral
│   ├── perceptron.py    # класс Perceptron
│   ├── metrics_utils.py  # confusion matrix, classification report
│   ├── run_experiments.py # запуск экспериментов по всем распределениям
│   ├── circles.png       # визуализация circles
│   ├── xor.png           # визуализация xor
│   ├── blobs.png        # визуализация blobs
│   └── spiral.png        # визуализация spiral
└── ...
```

7. Приложение: параметры экспериментов в playground.tensorflow.org

Dataset	Layers	Activation	Train size	Noise	Комментарий
Circles	8-8-8	tanh	10%	8%	переобучение выражено
XOR	10-10-10	relu	10%	12%	сложная граница
Blobs	12-12-12	tanh	5%	15%	запоминание флуктуаций
Spiral	16-16-16	tanh	10%	10%	максимальный разрыв train/ test

Таблица 4. Пример конфигураций, при которых наблюдается переобучение.

В отчёт могут быть добавлены скриншоты из playground для каждого случая (граница классов, кривые loss, значения train/test loss в конце обучения).